TTS Narrator Guide

베리어프리 키오스크의 내레이터 음성 안내 구현 가이드입니다.

Quick Start

aria-label 속성에 발화를 원하는 텍스트를 넣으면 됩니다.
tabIndex로 포커스 가능한 요소에 aria-label을 작성하면, 해당 요소가 포커스될 때 텍스트가 자동으로 음성 변환됩니다.

가장 기본적인 사용법 TypeScript

```typescript
// tabIndex로 포커스 가능하게 만들고, aria-label에 읽어줄 텍스트를 넣습니다.
<button
  tabIndex={0}
  aria-label="층별안내: 확인 버튼을 누르시면 층별 안내로 이동합니다."
>
  층별안내
</button>
```

이것이 전부입니다. `tabIndex` 로 포커스 가능한 요소에 포커스가 들어오면, `aria-label` 에 작성된 텍스트가 음성으로 재생됩니다.

```
tabIndex 로 포커스 가능 → 요소에 포커스 진입 → aria-label 텍스트 추출 → TTS 음성 재생
```

TTS 엔진	포커스된 요소의 aria-label 처리 방식
Windows 내레이터	`tabIndex` 로 포커스된 요소의 `aria-label` 을 Windows 스크린 리더가 자동으로 읽음
로컬 TTS (ONNX)	`tabIndex` 로 포커스된 요소의 `aria-label` 텍스트를 추출하여 로컬 TTS 엔진에 전달 후 음성 생성

개발자가 하는 일은 `tabIndex` 와 `aria-label` 을 작성하는 것뿐이고, 나머지는 TTS 엔진이 자동으로 처리합니다.

동작별 예시

키오스크 화면의 각 동작에 맞는 `aria-label` 작성 예시입니다. 그대로 복사하여 사용할 수 있습니다.

메뉴 버튼

메뉴 버튼 TypeScript

```
<button aria-label="층별안내: 확인 버튼을 누르시면 층별 안내로 이동합니다.">
  층별안내
</button>

<button aria-label="행사안내: 확인 버튼을 누르시면 행사 안내로 이동합니다.">
  행사안내
</button>
```

선택 상태가 있는 버튼

동적 상태 반영 TypeScript

```
const MenuButton = ({ name, selected }: { name: string; selected: boolean })
  <button
    aria-label={`${name}${selected ? ', 선택됨' : ''}: 확인 시 이동합니다.`}
  >
    {name}
  </button>
);

// selected=true  → "층별안내, 선택됨: 확인 시 이동합니다."
// selected=false → "층별안내: 확인 시 이동합니다."
```

시설 목록

리스트 렌더링 TypeScript

```
interface Facility {
  id: string;
  floor: number;
  name: string;
}

const FacilityList = ({ items }: { items: Facility[] }) => (
  <>
    {items.map((item) => (
      <div
```

```
        key={item.id}
        aria-label={`${item.floor}층 ${item.name}, 확인 시 길 안내를 시작합니다.`}
        tabIndex={0}
      >
        {item.name}
      </div>
    ))}
  </>
);

// → "3층 대회의실, 확인 시 길 안내를 시작합니다."
```

닫기 / 네비게이션

네비게이션 버튼 TypeScript

```
<button aria-label="닫기 버튼. 확인 시 이전 단계로 돌아갑니다.">
  <CloseIcon />
</button>

<button aligned="다시보기, 확인 시 안내를 다시 시작합니다.">
  다시보기
</button>

<button aria-label="처음으로, 확인 시 메인 화면으로 돌아갑니다.">
  처음으로
</button>
```

초기 안내 가이드

가이드 메시지 TypeScript

```
const guideLabel = [
  '안녕하세요. 키오스크 사용을 도와드리겠습니다.',
  '상하좌우 방향키로 항목을 이동할 수 있습니다.',
  '확인 버튼을 누르면 선택됩니다.',
].join(' ');

<div aria-label={guideLabel} tabIndex={0}>
  키오스크 안내
</div>

// → "안녕하세요. 키오스크 사용을 도와드리겠습니다. 상하좌우 방향키로 항목을 이동할 수 있습니다. ≡
```

label 생성 함수 분리

label 텍스트가 복잡하거나 여러 곳에서 재사용된다면 함수로 분리할 수 있습니다.

utils/labels.ts TypeScript

```typescript
export const floorLabel = (floor: { name: string; facilities: string }): str
  `${floor.name} 안내입니다. 이 층에는 ${floor.facilities}이 있습니다.`;

export const menuLabel = (name: string, selected: boolean): string =>
  `${name}${selected ? ', 선택됨' : ''}: 확인 시 이동합니다.`;
```

사용 TypeScript

```typescript
import { floorLabel, menuLabel } from '../utils/labels';

<div aria-label={floorLabel(currentFloor)}>...</div>
<button aria-label={menuLabel('층별안내', true)}>층별안내</button>
```